

22. Malom

Készítette:

Asztalos Gergely
su3ax9@inf.elte.hu

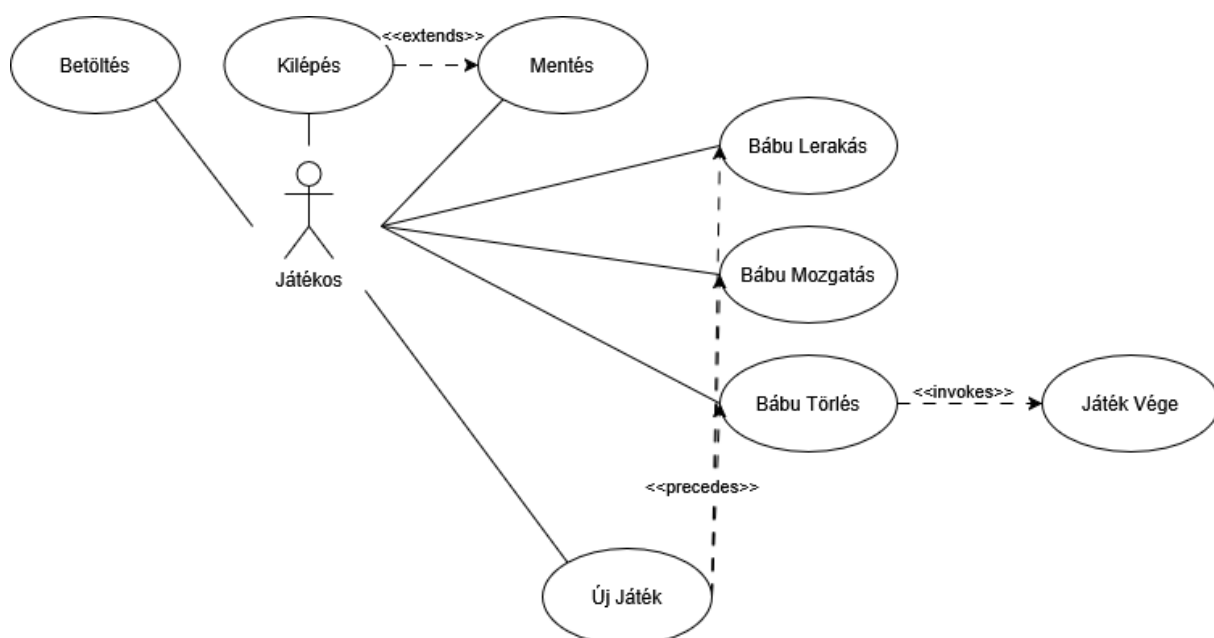
Feladat:

Készítsünk programot, amellyel a következő két személyes játékot játszhatjuk. A malom játékban két játékos egy 24 mezőből álló speciális játéktáblán játszhatja, ahol a mezők három egymásba helyezett négyzetben helyezkednek (mindegyikben 8, a sarkoknál és a felezőpontoknál), melyek a felezőpontok mentén össze vannak kötve. Kezdetben a tábla üres, és felváltva helyezhetik el rajta bábuikat az üres mezőkre. Az elhelyezés után a játékosok felváltva mozgathatják bábuikat a szomszédos (összekötött) mezőkre. Amennyiben egy játékos nem tud mozgatni, akkor passzolhat a másik játékosnak. Ha valakinek sikerül 3 egymás melletti mezőt elfoglalnia (azaz malmot alakít ki, rakodás, vagy mozgatás közben), akkor leveheti az ellenfél egy általa megjelölt bábuját (kivéve, ha az egy malom része). Az a játékos veszít, akinek először megy 3 alá a bábuk száma a mozgatási fázis alatt.

Elemzés:

- A játék menete során a két játékos felváltva cselekszik, három lehetséges akciója van:
 - A táblára lehelyez egy bábut, amíg mindkét játékos 9 bábut le nem helyezett.
 - Malom esetén (3 bábu egy sorban/oszlopban) leveheti az ellenfél egy bábuját.
 - Az összes bábu lehelyezése után a körében mozgathat bábut egy szomszédos mezőre.
- A feladatot egyablakos asztali alkalmazásként Windows Forms grafikus felülettel valósítjuk meg.
- Lenyíló menüben választhatunk különböző opciók közül: Új játékot kezddhetünk, az aktuális mezőt elmenthetjük, illetve betölthetünk már létező mentést. Ezen felül ki is léphetünk, ekkor a játék felugró mezővel lehetőséget kínál az állapot mentésére.
- A játéktábla a tipikus malomjáték elrendezését követi (három négyzet egymásban, a felezőpontoknál összekötve). Adott mezőre kattintva az aktuálisan valid akciónak megfelelő esemény következik be (Lerakunk egy bábut, mozgatunk, vagy levesszük ellenfelünk bábuját).
- A program implementálja a malomjáték specifikus szabályait:

- o Ellenfél malmából csak akkor vehetünk le bábut, ha nincs több malomba nem tartozó bábuja a táblán.
- o Ugrás-szabály: Ha csupán 3 bábuk maradt vissza (1 további bábu elvesztése vereséggel járna), akárhova léphetünk a szomszédos mezők helyett.
- o Körönként csak egy malom "érvényes", tehát hiába alakítunk ki két malmot, csak egy bábut vehetünk le a tábláról.
- A játék végén (amikor valamely játékosnak 7 bábuja is lekerült a tábláról), dialógus értesít minket a győztes játékosról, és lehetőséget ajánl új játék indítására.
- A felhasználói esetek az 1. ábrán láthatóak.



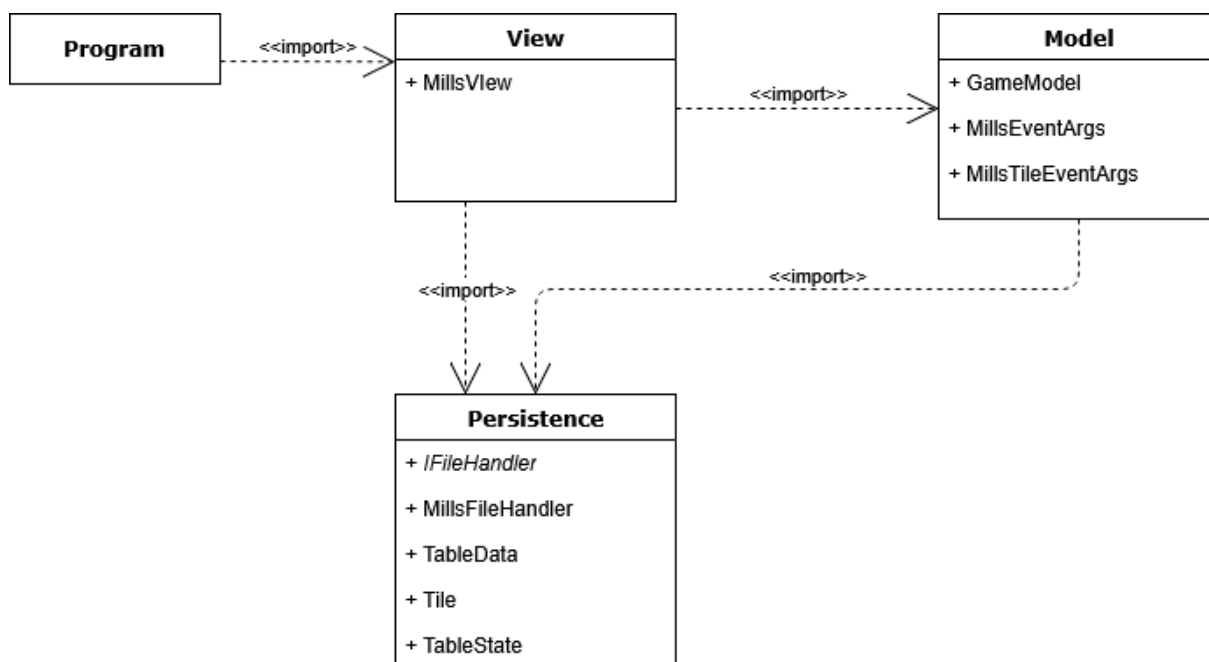
1. ábra: Felhasználói esetek diagramja

Tervezés:

- Programszerkezet:
- A programot háromrétegű architektúrában valósítjuk meg. A megjelenítés a **View**, a modell a **Model**, míg a perzisztencia a **Persistence** névtérben helyezkedik el. A program csomagszerkezete a 2. ábrán látható.
- A program szerkezetét két projektre osztjuk implementációs megfontolásból: a **Persistence** és **Model** csomagok a program felületfüggetlen projektjében, míg a **View** csomag a Windows Formstól függő projektjében kap helyet. A tesztesetek külön névtérben helyezkednek el.

Perzisztencia:

- Az adatkezelés feladata a játék állapotának tárolása.
- A **TableData** osztály a Malomjáték tábláját ábrázolja:
 - o A benne lévő **_tiles** tomb a pálya gráfos megjelenítését implementálja. Egy **tile** az alábbiképp van definiálva: eltároljuk a játékos, akinek bábuja a mezőn fekszik, illetve a szomszédos mezőket.
 - o Biztosít metódusokat a modell számára a pályát lévő lépések megvalósítására, mint a **SetTile()**, **ClearTile()**, **Move()**; illetve a különböző feltételek ellenőrzésére: van-e malmunk adott mezőben (**HasMill**), egy mezőt eltávolíthat-e a játékos, ha malma van (**IsRemovable**).
 - o A transzparencia és egyszerű bővíthetőség érdekében a játékosokat (**Player**) és a szomszédsági irányokat (**Direction**) enum-okkal adjuk meg. Ezekre az enum-okra a modell is hivatkozni fog.
- A játékállapot eltárolására szolgáló metódusokat az **IFileHandler** interfész határozza meg, a **TableState** struktúra segítségével történik az adatátadás az állapot mentése (**SaveFile**) és betöltése (**LoadFile**) céljából.
- Egy implementációja az **IFileHandler** interfésznek a **MillsFileHandler** osztály, amely szöveges fájlokba képes írni az adatokat, illetve azokból kiolvas egy korábbi játékállapotot. Az átláthatóság érdekében a fájlkezelés **.nmm** kiterjesztésű fájlokkal történik.
- A fájl első sorában adott a jelenlegi kör, a soron lévő játékos, illetve hogy hány bábút veszített egy-egy játékos. Következőnek soronként szerepelnek a pálya mezői, melyik játékos helyezett rájuk bábút: A tábla ábrázolásához hasonlóképpen, üres mező esetén **"Empty"** szerepel az adott sorban.



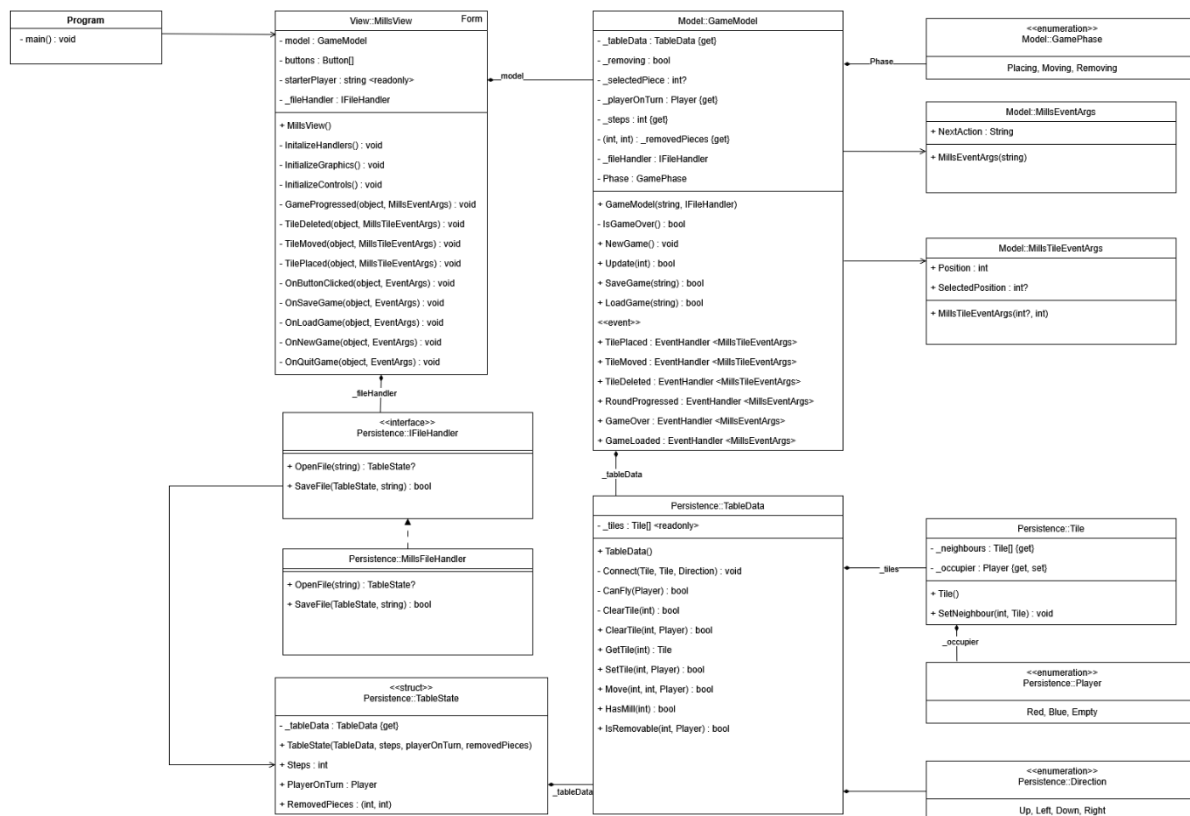
2. ábra: Az alkalmazás csomagdiagramja

Modell:

- A modellt a **GameModel** osztály szabja meg. Eltárolja az aktuális pályát, illetve a játék nyomkövetéséhez szükséges adatokat: éppen töröl-e a soron lévő játékos bábút (**_isRemoving**), van-e mozgatható kijelölt bábu - és ha igen, melyik - (**_selectedPiece**), a soron következő játékos (**_playerOnTurn**), a jelenlegi kör (**_steps**), játékosonként a már leütött bábuk száma (**_removedPieces**).
- A különböző akciókat három esemény kezeli le: **TilePlaced**, **TileMoved**, **TileDeleted**. Az esemény argumentuma minden esetben (**MillsTileEventArgs**) eltárolja, éppen melyik mezőt éri az adott változás (**Position**), továbbá megadja, volt-e előzőleg mező kijelölve (**SelectedPosition**) – ez a mozgatható során válik hasznunkra.
- A játékalapot előrehaladtáról a **GameProgressed** esemény kiváltásával jelzünk a nézetnek, a végéről pedig a **GameOver** eseménnyel. Új játék kezdetekor szintén a **GameOver** kerül meghívásra, azonban ekkor értelemszerűen nincs nyertes.
- Amennyiben korábbi játékot töltünk be, sikeres betöltés esetén a **GameLoaded** esemény kerül kiváltásra.
- A modell példányosításkor konstruktorban megkapja az adatkezelés egy konkrét implementációját a **_fileHandler** adattagként, így a **SaveGame** és **LoadGame** meghívják a fájlkezelő metódusait.
- A modell konstruktorja ezen felül a kezdő játékot is paraméterül kapja, a kezdő játékot ezen érték szerint állítja be.

Nézet:

- A nézetet a **MillsView** osztály biztosítja, amely tárolja a modell egy példányát (**model**).
- A modell táblája áttetsző gombokból áll, a tábla kinézetét egy statikus háttérkép adja. A felületen megfelelő menüpontok vannak, hozzájuk eseménykezelők tartoznak, melyek meghívják a modell metódusait. Vizuális ablakok biztosítják a felhasználóbarát alkalmazást, lenyíló ablakban választhatunk az opciók közül.
- A program teljes statikus szerkezete a 3. ábrán látható.



3. ábra: Az alkalmazás osztálydiagramja

Tesztelés:

- A modell funkcionalitása egységtesztek segítségével lett ellenőrizve a **MillsModelTests** osztályban.
- Az adatkezelés egy **IFileHandler** interfészből származtatott, mockolt fájlkezelő implementációval valósul meg.

Az alábbi tesztesetek kerültek megvalósításra:

- **NewGameTest**: Új játék indítása, kezdő állapotok helyességének vizsgálata.
- **PlayerStepTest**: Néhány lépés során a pálya állapotváltozásának ellenőrzése.
- **GameMillTest**: Malom kialakítása után a játék fázisa **Removing**-ba lép, ismét a malmot kialakító játékos jön. Ellenfél bábuja eltávolításának helyessége.
- **GameLoadTest**: Mockolt perzisztenciával játék betöltése, betöltést követően a mezők helyes állapotba kerülnek.
- **GameSaveTest**: Mockolt perzisztenciával játék elmentése, a mentendő állapot helyességének ellenőrzése.